

Data Scientist → Senior Data Scientist

A 9-12 Month Upskilling Plan: Projects, Open Source & Business Impact

Prepared for: A Data Scientist with 2.5 years of experience, targeting Classical ML/Analytics, LLMs, Deep Learning and Business Problem Solving, at a pace of 3-5 hours/week.

1. What 'Senior' Actually Means (So You're Aiming at the Right Target)

Moving from mid-level to Senior Data Scientist is rarely just "knows more algorithms." Hiring panels and promotion committees typically look for four things at once:

- **Technical depth** — you can go beyond calling `.fit()`: you understand trade-offs, debug models, and know when NOT to use ML.
- **Business framing** — you can translate a vague ask ("sales are down") into a measurable problem, and translate model output back into a decision.
- **Ownership & independence** — you can take a problem from ambiguity to a shipped/communicated result with minimal hand-holding.
- **Influence beyond your own code** — through code reviews, mentoring, writing, or open-source contributions that show you can operate in someone else's codebase and standards.

This plan is built around all four — not just "more projects." Every project below is chosen to double as a portfolio piece, a skill-builder, and a talking point for interviews or promotion conversations.

2. Your Reality: 3-5 Hours / Week

At 3-5 hrs/week (~15-20 hrs/month), the plan below spans roughly **9-12 months**. That is intentional — shallow, rushed projects hurt your portfolio more than they help. One well-documented project beats five half-finished repos.

Weekly Time Block	Suggested Use
Mon/Wed evening (1-1.5 hr each)	Learning: courses, papers, docs, reading source code
Weekend block (2-2.5 hr)	Hands-on: project work OR an open-source PR
15 min, any day	Log progress in a build log / journal (see Section 6)

Golden rule: alternate — don't try to learn Deep Learning, build a RAG app, AND contribute to open source in the same week. Sequence it (Section 3) so each phase gets real focus.

3. The Phased Roadmap

Phase 0 — Foundations & Setup (Weeks 1-2)

Goal: set up the infrastructure so every hour you spend from here on is visible and reusable.

- Create/clean up a **GitHub profile** with a proper README (pinned repos, skills, current focus).
- Set up a simple **build log** (Notion/Google Doc/markdown file) — 3 lines after every session: what you did, what you learned, what's next.
- Do a **skills gap self-audit** against 3-4 real Senior DS job descriptions in your target companies — note recurring keywords (causal inference, experimentation, LLM evals, MLOps, A/B testing, etc.).
- Pick a **public domain / theme** for your portfolio (e.g., "pricing & demand", "customer analytics", "fintech risk") so your projects feel like a coherent body of work, not random Kaggle notebooks.

Phase 1 — Classical ML, Analytics & Business Problem Solving (Months 1-3)

This is your foundation and likely your fastest path to "senior-sounding" — because most interview loops and real jobs are won or lost on business framing, not exotic architectures.

- **Sharpen the fundamentals:** revisit feature engineering, leakage, imbalanced data, model evaluation beyond accuracy (precision/recall trade-offs, calibration, cost-sensitive metrics), and experimentation design (A/B testing, power analysis, guardrail metrics).
- **Learn a structured problem-solving framework:** practice converting an ambiguous prompt ("reduce churn", "improve conversion") into a problem statement, success metric, and hypothesis list before touching data — this is a core "senior" signal.
- **Flagship Project 1 — End-to-End Business Case:** pick a real dataset (e.g., e-commerce, telecom churn, retail demand) and build a full pipeline: business problem → EDA → feature engineering → 2-3 candidate models → evaluation tied to a business metric (revenue saved, cost avoided) → a one-page executive summary + a technical write-up. Deploy it as a small Streamlit/Gradio app so it's demoable, not just a notebook.

Why this matters: a project that ends in "and this would save ~\$X or improve Y by Z%" reads as senior. A project that ends in "and I got 92% accuracy" reads as junior.

Phase 2 — Deep Learning Foundations (Months 3-5)

Goal: enough depth that you can reason about architectures, not just import them.

- Rebuild core intuition: backprop, CNNs, RNNs/attention, optimization (why Adam, why batch norm), and where classical ML still beats DL (small/tabular data).
- Get comfortable with **PyTorch** at a level where you can write a custom training loop, not just call `.fit()`.
- **Flagship Project 2 — Applied Deep Learning:** a focused project in an area adjacent to your work (e.g., image-based defect/quality detection, demand forecasting with a temporal deep net, or a recommendation system). Include an honest comparison: "here's what a simpler classical model got, here's what DL added, here's the cost/complexity trade-off." That comparison is itself a strong senior-level artifact.

Phase 3 — LLMs & Applied GenAI (Months 5-7)

This is the fastest-growing and most visible skill area right now — but avoid "just calling the OpenAI/Anthropic API" and stopping there.

- Learn the landscape: embeddings, vector search, RAG architecture, prompt engineering, evaluation of LLM outputs (this is where most people stop), and lightweight fine-tuning/LoRA concepts.
- Learn **LLM evaluation** specifically — building a small eval set, scoring rubrics, catching hallucinations — this is a skill most "prompt engineers" skip and it's exactly what makes an LLM project look senior.
- **Flagship Project 3 — Domain RAG / GenAI Assistant:** build a retrieval-augmented assistant over a non-trivial corpus (company policies, product docs, financial filings, or your own Phase-1 project's documentation). Include: chunking strategy notes, retrieval quality metrics, a small hand-labeled eval set, and a cost/latency discussion. Ship a simple UI.

Phase 4 — Open Source Contributions (Months 3-9, running in parallel)

Start this in parallel with Phase 1, not after everything else — open source is a slow-burn credibility signal and early, small contributions compound. See Section 4 for the detailed strategy.

Phase 5 — Capstone, Portfolio Polish & Positioning (Months 9-12)

- Pick your strongest 1-2 projects and polish them: clean READMEs, architecture diagrams, a short demo video/GIF, and a written case study (blog post or LinkedIn article).
- Write a short **"Impact & Growth" document** for yourself — mapped against the Senior DS job descriptions from Phase 0 — to use in performance reviews or interviews.
- If applicable, start **mentoring or reviewing** junior teammates'/community members' code — this is one of the highest-leverage, most senior-coded behaviors and costs very little extra time.
- Consider a short talk/write-up at a local meetup, internal tech talk, or Medium/Substack post summarizing one flagship project — visibility compounds your credibility signal.

4. Open Source Contribution Strategy

Open source contributions are valued because they prove you can read unfamiliar code, follow someone else's standards, and get real reviewers to accept your work — a very different skill from a solo Kaggle notebook.

4.1 Where to Contribute (aligned to your focus areas)

- **Classical ML / Analytics:** scikit-learn, XGBoost, LightGBM, pandas, statsmodels, Great Expectations, Evidently AI
- **Deep Learning:** PyTorch (ecosystem repos, not core), timm, PyTorch Lightning, Albumentations
- **LLMs:** LangChain, LlamaIndex, Hugging Face (transformers/datasets), Haystack, Ragas (RAG evaluation)
- **Business / Viz / Tooling:** Streamlit, Plotly, MLflow, DVC

4.2 The Progression (don't start with a big feature)

- **Month 1-2:** Fix documentation, typos, add missing docstrings/examples. Low risk, gets you comfortable with the repo's contribution process (CLA, PR templates, CI checks).
- **Month 2-4:** Pick up issues labeled *good first issue* / *help wanted*. Fix small bugs, add unit tests for uncovered code.
- **Month 4-7:** Take on small feature requests or performance improvements. Start participating in issue discussions before someone assigns you — this shows initiative.
- **Month 7+:** Propose a small enhancement yourself (based on a gap you hit in your own projects), or become a repeat contributor/reviewer on a repo you know well.

4.3 Practical Search Tips

- Use GitHub's own search: `is:issue is:open label:"good first issue" language:python` filtered to your target repos.
- Sites like **goodfirstissue.dev** and **up-for-grabs.net** aggregate beginner-friendly issues.
- Read the repo's *CONTRIBUTING.md* fully before your first PR — most rejected PRs fail on process, not code quality.
- Comment "I'd like to work on this" on an issue before starting — avoids duplicate work and gets you visible early feedback.

Realistic target: at 3-5 hrs/week shared with project work, aim for 1 merged PR every 4-6 weeks. By month 9 that's roughly 6-8 merged PRs — a credible, honest open-source track record.

5. Making It All Visible (Portfolio & Tracking)

- **GitHub:** one pinned repo per flagship project, each with a proper README (problem, approach, results, how to run it) — this is what recruiters and panels actually skim.
- **Writing:** 1 blog/LinkedIn post per flagship project (600-900 words: problem → approach → result → what you'd do differently). This is often what gets you noticed before anyone opens your code.

- **Build log:** keep the weekly log from Phase 0 going the whole year — it becomes the raw material for your resume bullets, performance review, and interview stories (use it for STAR-format answers).
- **Resume/LinkedIn:** update quarterly with quantified outcomes ("reduced X by Y%", "merged N PRs to [project]", "built RAG system handling Z documents") rather than tool lists.

6. Milestone Checklist

- Month 1: Portfolio infra set up, first business-framing project problem statement written
- Month 2: First open-source PR submitted (docs/typo-level is fine)
- Month 3: Flagship Project 1 (Business/Classical ML) shipped with write-up
- Month 4: First 'real' open-source bug-fix PR merged
- Month 5: PyTorch fundamentals solid; Flagship Project 2 (Deep Learning) underway
- Month 6: Flagship Project 2 shipped with write-up
- Month 7: LLM/RAG fundamentals solid; Flagship Project 3 underway; 3+ PRs merged total
- Month 9: Flagship Project 3 shipped; 5-6+ PRs merged total
- Month 10-11: Portfolio polish, case-study writing, mentoring/reviewing started
- Month 12: Impact & Growth document ready; positioned for senior-level conversations

Note: this is a roadmap, not a rulebook — adjust phase order if your day job suddenly needs deep LLM or DL work sooner. The constants that matter most are: keep shipping documented work, keep a public trail (GitHub + writing), and keep tying every project back to a business outcome.